
INF1350

Sistemas Reativos

— Arduino Programação —

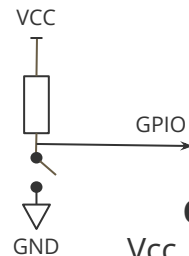
Prof. Adriano Branco

abranco@inf.puc-rio.br

Slides baseado em aulas anteriores da Prof. Noemi Rodriguez e com contribuição do Prof. Francisco Sant'Anna e do Prof. Adriano Branco

Programa com entrada e saída

```
...  
// função loop executa para sempre  
// (chamada de loop do framework)  
void loop() {  
    digitalWrite(LED1, LOW); // acender o led (voltagem em LOW)  
    delay(1000); // esperar um segundo  
    digitalWrite(LED1, HIGH); // apagar o led (voltagem em HIGH)  
    delay(1000); // esperar um segundo  
    if (!digitalRead(KEY1)) {  
        digitalWrite(LED1, LOW);  
        exit(0);  
    }  
}
```



Chave aberta
Vcc, +5v, 1, High, True

Chave fechada
GND, 0v, 0, Low, False

- pisca-pisca...
- parar quando usuário apertar a chave 1 e deixar a luz acesa para sempre?

programas reativos

- chamadas **bloqueantes** limitam reatividade
- alternativa: controle do tempo decorrido

Contagem de tempo, função *millis()*

millis()

Description

Returns the number of milliseconds passed since the Arduino board began running the current program. This number will overflow (go back to zero), after approximately 50 days.

Syntax

```
time = millis()
```

Returns

Number of milliseconds passed since the program started.

Data type: `unsigned long (32 bits)`.

<https://docs.arduino.cc/language-reference/en/functions/time/millis/>

controle do tempo no loop Arduino

```
int state = 1; // GLOBAIS MANTÊM ESTADO
unsigned long old;
```

```
void setup() {
```

```
  ...
```

```
  old = millis();
```

```
}
```

```
void loop () {
```

```
  unsigned long now = millis();
```

```
  if (now >= old+1000) { // hora de piscar?
```

```
    old = now;
```

```
    state = !state;
```

```
    digitalWrite(LED1, state);
```

```
  }
```

```
}
```

com reação ao pressionamento da chave:

```
int state = 1; // GLOBAIS MANTÊM ESTADO
unsigned long old;
...
void loop () {
    unsigned long now = millis();
    if (now >= old+1000) { // hora de piscar?
        old = now;
        state = !state;
        digitalWrite(LED1, state);
    }
    if (!digitalRead(KEY1)) { // verifica chave
        digitalWrite(LED1, LOW);
        exit(0);
    }
}
```

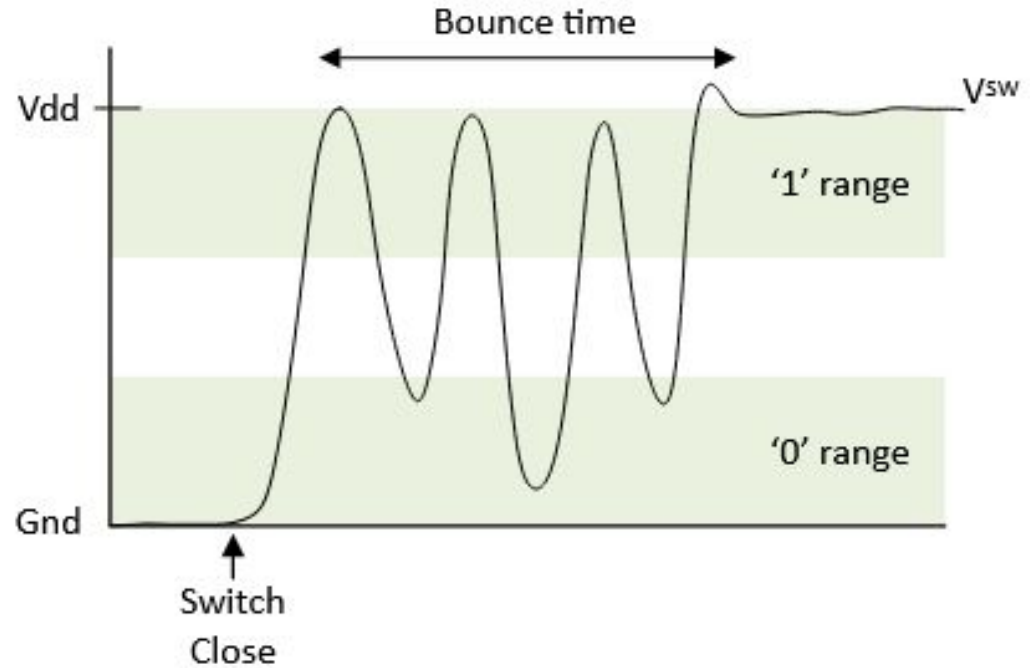
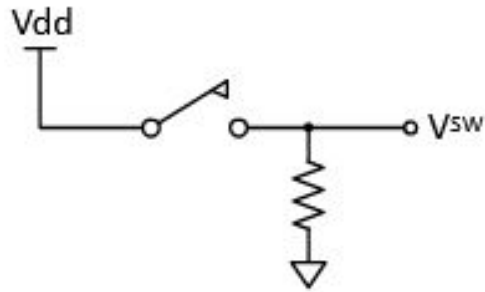
Agora é reativo, mas ainda “feioso” !!!!

outro exemplo com o loop Arduino

liga e desliga led pressionando uma chave

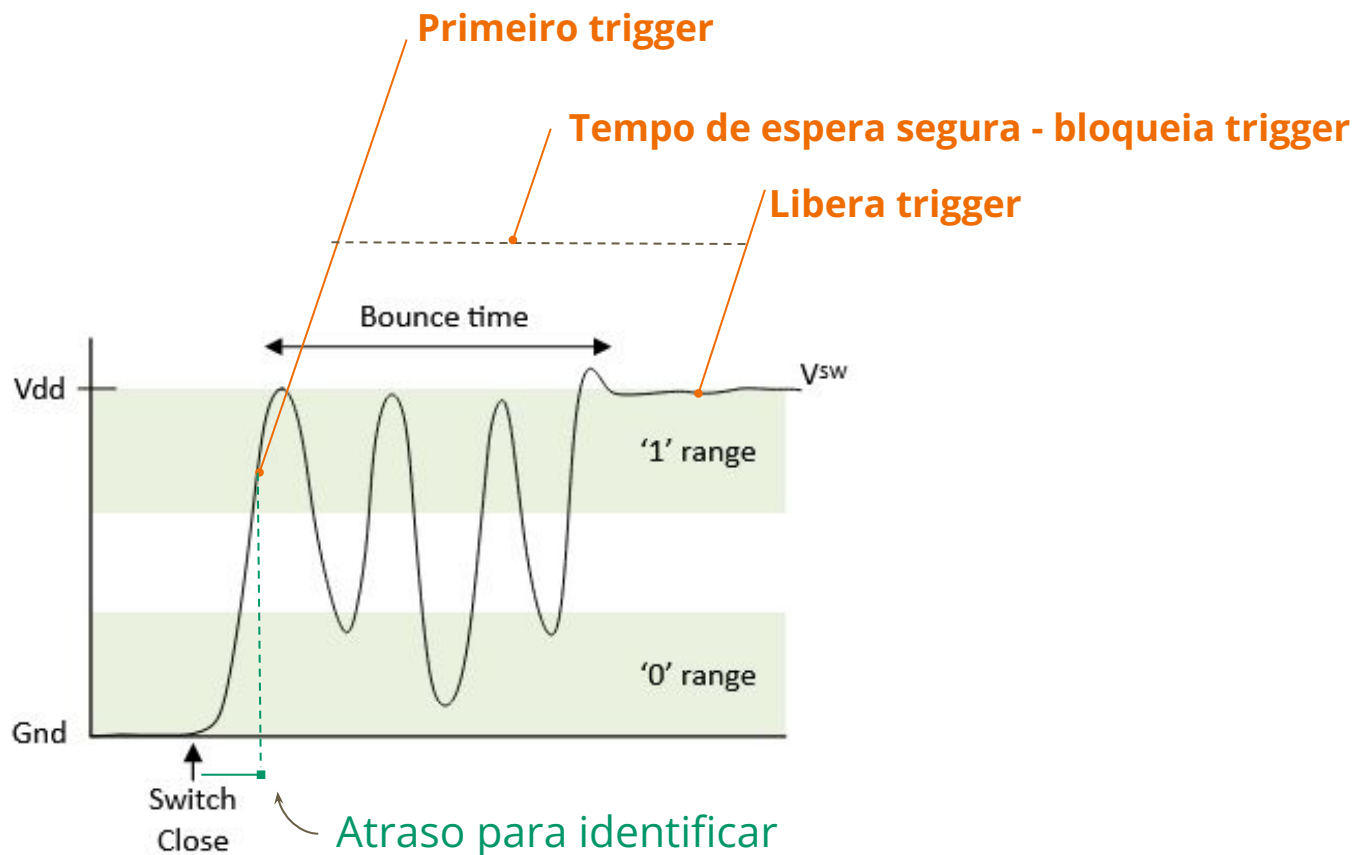
```
int ledOn = 0;
void setup() {
  // set digital pins:
  pinMode(LED1, OUTPUT);
  pinMode (KEY1, INPUT_PULLUP);
  digitalWrite(LED1, HIGH);
}
// pressionou a chave? mudar o estado do led!
// primeira tentativa:
void loop() {
  if (digitalRead(KEY1) == LOW) {
    ledOn = !ledOn;
    digitalWrite(LED1, ledOn);
  }
}
```

Switch Bounce



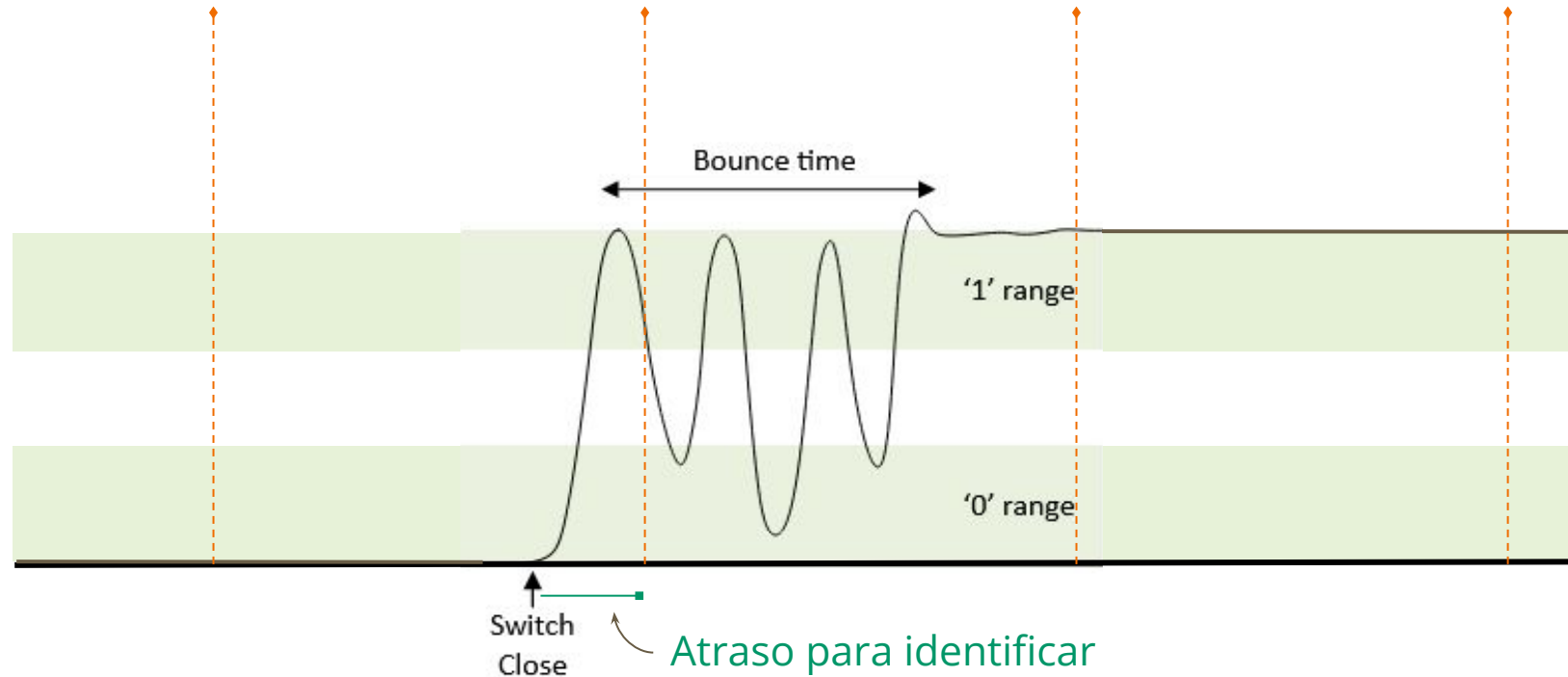
Debounce

Com trigger



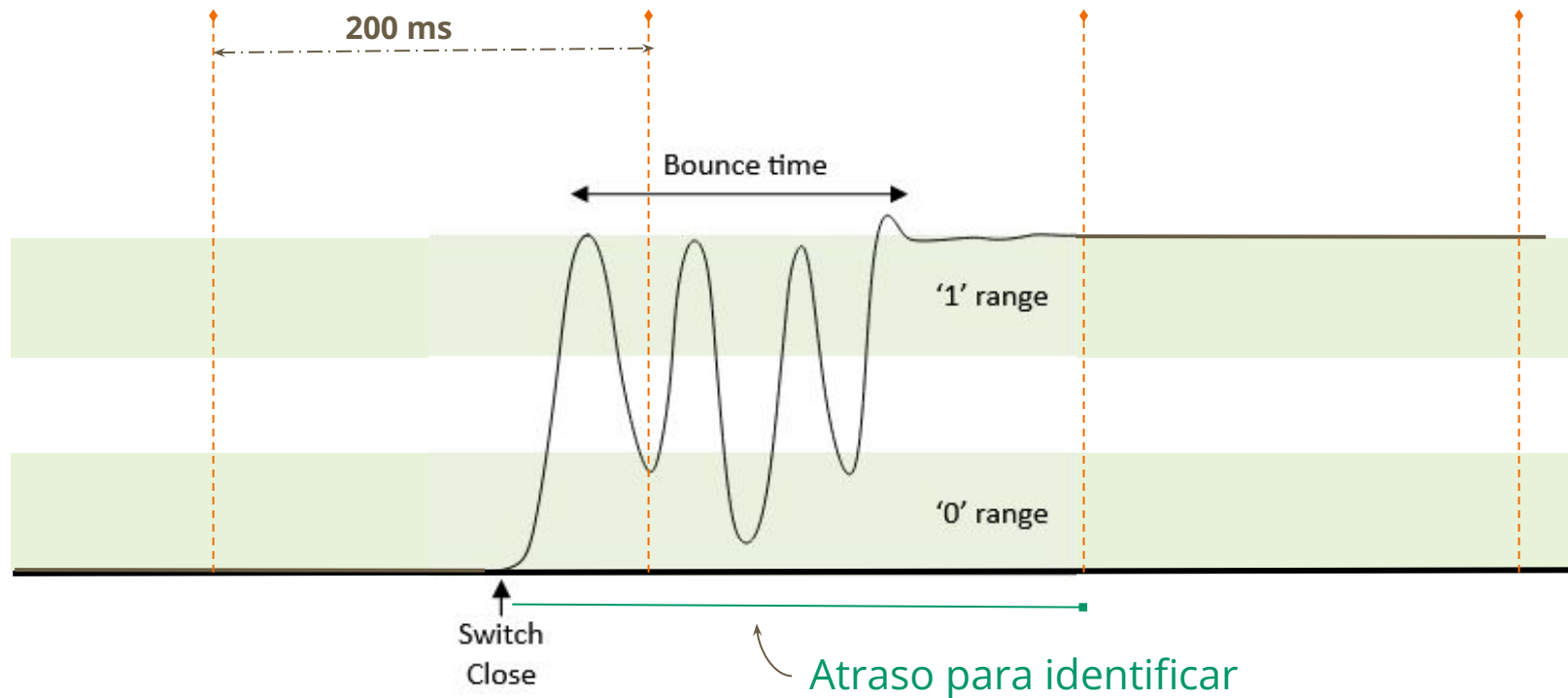
Debounce

Sem trigger - leituras periódicas (intervalo)



Debounce

Sem trigger - leituras periódicas (intervalo)



reconhecendo o pressionamento da chave

```
#include "pindefs.h"
int ledOn = 0;
unsigned long previousMillis = 0;
unsigned long interval = 200; // 200 ms
void setup() { ... }
void loop() {
    unsigned long currentMillis = millis();
    // if time enough has elapsed, check if the pushbutton is pressed.
    // if it is, the buttonState is LOW:
    if (currentMillis - previousMillis >= interval) {
        if (digitalRead(KEY1) == LOW) {
            previousMillis = currentMillis; // save the last time you read LOW
            ledOn = !ledOn;
            digitalWrite(LED1, ledOn);
        }
    }
}
```

reconhecendo o pressionamento da chave

- debounce: existem também soluções por hardware
- por software: calibragem do tempo

Exercícios

1. Transportar para o arduino o programa que liga e desliga o led quando se aperta a chave 1 (slide 11). Experimentar com diferentes intervalos de tempo em que se ignora a chave e observar o comportamento.
2. Fazer um programa que pisca o led 1. Ao pressionar a chave 1, o tempo da “piscada” deve ser dividido por 2. Ao pressionar a chave 2, o tempo da “piscada” deve ser dobrado. (Colocar limites inferiores e superiores no tempo de piscada.)
3. Estender o programa anterior para que o led 1 pare de piscar ou volte a piscar ao pressionarmos a chave 3.

Tarefas 1 e 2 - Entrega no EAD em 1 semana, final do dia.

1. Alterar o programa anterior para que o programa pare de piscar ou volte a piscar quando chave 1 e chave 2 forem pressionadas “simultaneamente” (chave 1+2 em menos de 500 ms).
2. Escrever um programa que inicialmente mostre uma sequência aleatória de 5 “piscadas” dos leds 1, 2 ou 3 e em seguida espere o usuário “repetir” a sequência usando as chaves 1, 2 e 3. Caso o usuário erre, o programa deve acender o led 1 e deixa-lo aceso. Caso o usuário acerte a sequência, o programa deve acender todos os leds e deixa-los acesos. Em qualquer caso, se o usuário apertar a chave 1 o programa deve voltar ao estado inicial.